

기술 선택 — "왜 이걸 골랐나"

Q 왜 SSR을 안 쓰고 CSR인가요?

검색 노출이 필요 없는 서비스고, 화면이 소켓 이벤트로 계속 바뀝니다. **서버가 미리 그려 줄 게 없습니다.** Next.js도 봤지만 강점인 SSR·SEO·파일 라우팅을 하나도 안 써서 뺐습니다.

Q HTTP로는 안 되나요? 왜 WebSocket인가요?

HTTP는 물어봐야 답합니다. 봇이 말하거나 남이 투표하면 **서버가 먼저 말을 걸어야** 합니다. 폴링으로도 되지만 지연과 요청 수가 늘어납니다.

Q 왜 Vercel이 아니라 Railway인가요?

Vercel은 **WebSocket 연결 상한이 5분**입니다. 저희 한 판은 그보다 깁니다.

Q 왜 Express인가요? Nest·Fastify는요?

필요한 게 실시간 소켓 서버 하나였습니다. 딱 그만큼 하는 걸 골랐고, 정적 파일 서빙까지 겸해서 **프로세스도 포트도 하나**입니다.

Q Redux·Zustand 같은 상태 관리는 왜 없나요?

클라이언트가 상태를 소유하지 않기 때문입니다. **서버가 보낸 GameState 하나가 유일한 진실**이고, 화면은 그걸 그리는 함수입니다.

Q 라우터는 왜 안 썼나요?

URL이 두 번째 진실이 되면 **새로고침·뒤로가기**가 진행 중인 판을 깹니다. state.phase 하나로 화면을 가릅니다.

데이터 · DB

Q 왜 PostgreSQL인가요? NoSQL은요?

설문의 "어느 발언이 봇 같았나" 값이 **발언 기록을 정확히 가리켜야** 합니다. 표 사이 연결이 자가개선의 핵심이라 관계형을 썼습니다.

Q Supabase는 뭔가요?

PostgreSQL을 대신 운영해 주는 서비스입니다. **DB 종류는 PostgreSQL**, 호스팅하는 곳이 Supabase입니다.

Q 데이터는 얼마나 쌓였나요?

테스트 판 수 · 설문 응답 수 · 발언 수 — 발표 직전 실측해서 채울 것.

봇 · 자가개선 루프

Q 봇이 진짜 스스로 개선되나요? 사람은 뭘 하나요?

다섯 판이 쌓이면 루프가 스스로 깨어납니다. 실패 장면 확인 → 기준선 측정 → 지침 수정 → 재측정 → 설문 갱신 → PR. **사람이 하는 건 마지막 승인** 하나입니다.

Q 왜 완전 자동으로 안 했나요?

사람 검토 없이 프롬프트가 바뀌는 안(설문 자동 주입)은 **구현까지 갔다가 뺐습니다.** 나온 결과를 그냥 믿어야 하는 게 문제였습니다.

Q 개선됐다는 걸 어떻게 증명하나요?

고치기 전 상태를 반복 측정해 **기준선**을 잡고, 같은 상황에서 다시 재 전후를 비교합니다. 게다가 그 지점을 겨냥해 설문 문항이 갱신돼서 **다음 참가자가 다시 확인**해 줍니다.

Q 어떤 모델을 쓰나요?

Anthropic 호환 · OpenAI 호환(OpenRouter)을 **설정으로 교체**합니다. 게 임마다 어떤 커밋·어떤 모델로 돌았는지 DB에 남깁니다.

Q 여러 모델을 비교한 결과가 있나요?

아직 없습니다. 지금 구조에서 바로 할 수 있다는 단계까지 왔습니다 — 그래서 활용 방안을 벤치마크로 잡았습니다. (*모르는 건 모른다고*)

보안 · 안정성

Q F12를 열면 제시어가 보이지 않나요?

애초에 안 보냅니다. view.ts가 받는 **사람별로 상태를 잘라서** 보냅니다. 5명 이면 이 함수를 5번 돌립니다.

Q API 키는 어떻게 관리하나요?

LLM 키·Webhook URL·DB 키 모두 **서버에서만** 읽습니다. 브라우저 번들에 들어가지 않습니다.

Q 서버가 이상한 값을 보내면 화면이 죽지 않나요?

타입은 빌드가 끝나면 사라지므로 **진입점에서 한 번 더 확인**합니다. 그래도 터 지면 최상위 ErrorBoundary가 받아 흰 화면 대신 돌아올 길을 남깁니다.

프론트엔드 구현

Q 페이지가 8개인데 화면은 왜 1개인가요?

여덟 개를 비교하니 실제로 다른 건 **네 가지**(채팅 잠금 · 투표 패널 · 지목 배너 · 헤더 문구)뿐이었습니다. 순수 함수 modalFor가 판단하고 MainScreen 하나 그립니다.

Q 모바일 대응은 했나요?

PC 창을 줄여 만든 모바일과 **실제 폰은 달랐습니다.** 배포 후 실기기에서 채팅 입력 지연을 잡았고, 원인은 이벤트마다 목록 전체를 다시 그리던 것이었습니다.

한 줄로 잇기 — 막혔을 때

Vite가 만든 화면을 Express가 내보내고, React가 그린 화면이 Socket.IO로 서버와 계속 이야기하고, 그 결과가 PostgreSQL에 남습니다. 전부 Railway 하나 위에 올라가 있습니다.

모르면 "그건 확인해 보고 말씀드리겠습니다"가 지어내는 것보다 낫습니다. · **숫자는** 발표 직전에 다시 확인 — 커밋 수 · 파일 수 · 테스트 판 수. · **되돌아갈 문장** — "게임이 봇을 재고, 그 결과가 다시 게임으로 돌아갑니다."